

Taller 7: Despliegue con contenedores Docker en EC2

Instrucciones:

En este taller exploraremos el despliegue de soluciones analíticas usando contenedores Docker ejecutados en una máquina virtual de EC2.

La entrega de este taller consiste en un reporte en formato de documento de texto, donde pueda incorporar fácilmente capturas de pantalla, textos y elementos similares. El reporte debe entregarse en formato PDF.

Parte 1: despliegue de la API con contenedores en EC2

En esta primera parte usaremos una máquina virtual para desplegar allí nuestra API empleando un contenedor Docker.

1. En la consola de EC2 lance una instancia t3.micro, **Ubuntu server 24.04** con 20 GB de disco. **Incluya un pantallazo de la consola de AWS EC2 con la máquina en ejecución en su reporte. Su usuario de AWS y las IPs privada y pública deben estar visibles en el pantallazo.**
2. Para conectarse a la instancia, en una terminal emita el comando

```
ssh -i /path/to/llave.pem ubuntu@IP
```

donde */path/to/* se refiere a la ubicación del archivo *llave.pem* que descargó, e IP es la dirección IP pública de la instancia EC2 que lanzó. Si prefiere, en la terminal puede navegar a la ubicación del archivo llave.pem y emitir el comando

```
ssh -i llave.pem ubuntu@IP
```

Note que usamos en este caso ubuntu en vez de ec2-user, pues éste es el usuario creado por defecto como administrador con sistema operativo Ubuntu server.

3. Ahora pasamos a instalar Docker en la máquina, para lo cual requerimos eliminar posibles versiones anteriores y agregar el repositorio de la última versión estable de Docker.
4. En la máquina virtual, elimine versiones anteriores de Docker (esto genera un error si no hay versiones anteriores, en cuyo caso puede continuar sin problema) docker-doc podman-docker

```
sudo apt-get remove docker docker-engine docker.io containerd runc docker-doc  
podman-docker
```

5. Actualice el índice de paquetes

```
sudo apt-get update
```

6. Instale dependencias para verificar certificados (ca-certificates), obtener objetos con su URL (curl) y administrar llaves PGP (gnupg)

```
sudo apt-get install ca-certificates curl gnupg
```

7. Agregue la llave de Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor  
-o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

8. Agregue el repositorio de Docker a su sistema para la instalación

```
echo \  
"deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings/  
docker.gpg] https://download.docker.com/linux/ubuntu \  
"$(. /etc/os-release && echo "$VERSION_CODENAME)" stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

9. Actualice nuevamente el índice de paquetes con este nuevo repositorio incluido

```
sudo apt-get update
```

10. Instale Docker Engine, containerd, y Docker Compose

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-  
plugin docker-compose-plugin
```

docker-doc podman-docker

11. Para verificar su instalación, descargue, construya y ejecute la imagen hello-world

```
sudo docker run hello-world
```

Copie y explique la salida en pantalla en su **reporte**.

12. Adjunto a este enunciado encontrará el archivo *docker-api-starter.zip*. Descomprima el archivo **localmente** en una carpeta, que llamaremos raíz, tal que allí quede la carpeta bankchurn-api y el archivo Dockerfile. En esta carpeta raíz cree un repositorio git, y conéctelo con un repositorio nuevo en GitHub. Al terminar debe tener en su carpeta raíz 2 carpetas, .git y bankchurn-api, así como el archivo Dockerfile.
13. Explore el archivo Dockerfile y en su reporte explique brevemente el paso a paso de la configuración definida en este archivo. Para esto tenga presente que

- FROM determina la imagen base que se usa como sistema operativo y aplicaciones iniciales.
 - RUN ejecuta comandos al interior del contenedor.
 - COPY copia archivos del sistema hospedador (la máquina virtual) al contenedor.
 - ENV permite definir variables de entorno.
 - EXPOSE abre un puerto del contenedor.
 - CMD es el comando que se ejecuta al lanzar el contenedor.
14. Regrese a la terminal de su máquina virtual en EC2 y clone allí el repositorio. Ingrese a la carpeta del repositorio, donde debe encontrar la misma estructura de carpetas (.git, bankchurn-api) y el Dockerfile.
 15. A partir del archivo Dockerfile construya la imagen que usará más adelante para lanzar contenedores

```
sudo docker build -t bankchurn-api:latest .
```

Note que el comando termina con un espacio y un punto. Esto indica que se usa la carpeta actual como base para construir la imagen.
 16. Liste las imágenes de docker con el comando

```
sudo docker images
```

Debe contar con la imagen recién creada de bankchurn-api y la hello-world creada anteriormente. Incluya un pantallazo de la salida en su **reporte**. Su IP privada debe ser visible.
 17. Ahora ejecute un contenedor usando la imagen creada con el comando

```
sudo docker run -p 8001:8001 -it -e PORT=8001 bankchurn-api
```

Incluya un pantallazo de la salida de este comando en su **reporte**. La IP privada debe ser visible.
 18. Note que aquí conectamos el puerto 8001 del contenedor con el puerto 8001 de la máquina virtual (-p), dejamos activa la terminal interactiva (-it) y además definimos una variable de entorno PORT con el valor 8001, que se requiere al ejecutar el contenedor (-e). Para clarificar esto revise el archivo run.sh en la carpeta bankchurn-api y note que allí se usa una variable de entorno PORT.
 19. Vaya ahora a la consola de EC2, seleccione su máquina virtual y modifique el grupo de seguridad para permitir tráfico por el puerto 8001. Es decir, en el grupo de seguridad de la máquina edite las reglas de entrada y agregue una que permita tráfico por el puerto TCP 8001 desde cualquier IP (anywhere IPv4).
 20. Copie la IP pública de su máquina y en un navegador local visite la página IP:8001. Allí debe aparecer la API de bankchurn en ejecución. Incluya un pantallazo del navegador en su **reporte**. La dirección (IP pública) debe ser visible.

Parte 2: despliegue del tablero con contenedores en EC2

Ahora usaremos la misma máquina para desplegar allí un tablero empleando un contenedor Docker. También podría hacerse desde una segunda máquina, o incluso desde un entorno local, pero aquí lo haremos usando la misma máquina, aprovechando que ya ha quedado configurada con Docker.

1. Conéctese a la máquina usando otra terminal y el comando SSH.
2. Adjunto a este enunciado encontrará el archivo *docker-dash-starter.zip*. Descomprima el archivo **localmente** en una carpeta, que llamaremos raíz, tal que allí quede la carpeta app y el archivo Dockerfile. En esta carpeta raíz cree un repositorio git, y conéctelo con un repositorio nuevo en GitHub. Al terminar debe tener en su carpeta raíz 2 carpetas, .git y app, así como el archivo Dockerfile.
3. Localmente explore los archivos, especialmente app.py, run.sh y el Dockerfile. Note que en app.py se toman las variables de entorno API_URL y API_PORT para determinar a dónde enviar la solicitud a la API. El valor de estas variables lo incluiremos en el comando de ejecución del contenedor del tablero (ver más abajo).
4. Regrese a la terminal de su máquina virtual en EC2 y clone allí el repositorio. Ingrese a la carpeta del repositorio, donde debe encontrar la misma estructura de carpetas (.git, app) y el Dockerfile.
5. A partir del archivo Dockerfile construya la imagen que usará más adelante para lanzar contenedores
`sudo docker build -t bankchurn-dash:latest .`

Note que el comando termina con un espacio y un punto. Esto indica que se usa la carpeta actual como base para construir la imagen.

6. Liste las imágenes de docker con el comando

```
sudo docker images
```

Debe contar con la imagen recién creada de bankchurn-dash, además de la bankchurn-api y la hello-world creadas anteriormente. Incluya un pantallazo de la salida en su **reporte**. Su IP privada debe ser visible.

7. Ahora ejecute un contenedor usando la imagen creada con el comando

```
sudo docker run -p 8050:8050 -it -e PORT=8050 -e API_URL=X.Y.Z.W -e API_PORT=8001 bankchurn-dash
```

cambiando X.Y.Z.W por la IP pública de la máquina donde está corriendo la API. Incluya un pantallazo de la salida de este comando en su **reporte**. La IP privada debe ser visible.

8. Vaya ahora a la consola de EC2, seleccione su máquina virtual y modifique el grupo de seguridad para permitir tráfico por el puerto 8050. Es decir, en el grupo de seguridad de la máquina edite las reglas de entrada y agregue una regla que permita tráfico por el puerto TCP 8050 desde cualquier IP (anywhere IPv4).
9. Copie la IP pública de su máquina y en un navegador local visite la página IP:8050. Allí debe aparecer el tablero de bankchurn en ejecución.
10. Interactúe con el tablero y verifique que las solicitudes que se realizan se transmiten a la API y de allí de obtienen las predicciones que aparecen en el tablero. Incluya en su **reporte** un pantallazo donde aparezca la interfaz del tablero y las terminales de los dos contenedores en ejecución, mostrando los registros de la ejecución. Deben ser visibles la IP pública en el tablero y las IPs privadas en las terminales.

11. Compare los Dockerfile del tablero y de la API y comente en su **reporte** tres diferencias que considere sustanciales.

Rúbrica

A continuación se presenta la rúbrica de calificación, con un resumen de cada punto considerado. La descripción completa de cada instrucción la puede encontrar en el numeral asociado.

Numeral	Evidencia	Puntaje
1.1	Pantallazo de la consola de AWS EC2 con la máquina en ejecución	10
1.11	Pantallazo y explicación de salida de Docker hello-world	10
1.13	Explicación de Dockerfile	10
1.16	Pantallazo de salida de docker images	10
1.17	Pantallazo de salida de docker run	10
1.20	Pantallazo de navegador con API en ejecución	10
2.6	Pantallazo de salida de docker images	10
2.7	Pantallazo de salida de docker run	10
2.10	Pantallazo de tablero y terminales de los dos contenedores en ejecución	10
2.11	Comparación archivos Dockerfile	10